Global Rate Limiter Service - Project Overview

1. Project Description

This project is a scalable, microservice-based Global Rate Limiter. The system acts as a centralized rate-limiting engine for multiple external client apps. It exposes two core arithmetic endpoints (add and subtract) and restricts usage based on a strict limit of 1000 requests per client.

To ensure the core arithmetic API remains lightning fast, the rate-limiting checks and database updates are handled entirely asynchronously via a Celery background worker. This ensures a seamless and non-blocking user experience.

2. Tools and Technologies

- Web Framework: FastAPI (Python) - Exceptionally fast, modern, and automatically generates API documentation.
- Database: SQLite - A lightweight, file-based SQL database chosen specifically to meet the assessment requirements.
- ORM: SQLAlchemy - For secure, object-oriented database interactions and PEP8 compliance.
- Background Jobs: Celery - The industry standard for offloading heavy API tasks to background workers.
- Message Broker: Redis - An in-memory data store used by Celery to queue background tasks instantly.
- Containerization: Docker & Docker Compose - Ensures the entire microservice ecosystem runs reliably on the examiner's machine without requiring any complex local setup.

3. System Architecture

The system is divided into two distinct Microservices to enforce a strict separation of concerns:

1. Microservice 1 (Calculator & API Gateway):
- Handles incoming math requests (/add, /subtract).
- Before processing, it makes a quick GET request to Microservice 2 to check if the client is permitted.
- If permitted, it calculates the result and delegates the database usage update to the Celery Background Worker to prevent blocking the HTTP response.

2. Microservice 2 (Rate Limiter Engine):
- Solely responsible for interacting with the SQLite database.
- Tracks feature usage in the "Usage" table and updates the "Client_Status" table if a client exceeds the 1000 request limit.

- Mitigation: Due to SQLite's file-locking limitations under concurrent load, Write-Ahead Logging (WAL) mode was explicitly enabled in SQLAlchemy to support multiple background workers writing simultaneously.

3. Background Worker (Celery):
- Listens to the Redis queue for usage update events.
- Makes asynchronous PUT requests to Microservice 2 to update the database behind the scenes.

4. How to Run (For Examiners)

To evaluate and run the project, ensure you have Docker Desktop installed.

1. Open a terminal and navigate to the root directory (Global_Rate_Limiter_Service).
2. Start the entire ecosystem by running:
   docker-compose up --build

What will happen:
- Docker will build and spin up Microservice 1 (Port 8001), Microservice 2 (Port 8002), Redis, and the Celery Worker.
- A Client Simulation container will automatically start and simulate 3 concurrent clients making 1000, 1500, and 2000 requests.
- The simulator will generate text logs (e.g., client_2_logs.txt) inside the 'client' folder on your host machine.
- Open these text logs to clearly observe the system transitioning the clients from "Allowed" to "Exceeded" exactly at request 1001.

To stop the services and completely reset the database for a fresh test run, execute:
docker-compose down -v